



# Python Programming Fundamentals

## Properties — @property

Programming Fundamentals Team

SETU

2026-06-12



# Outline

1. The Problem with Getters and Setters
2. The @property Decorator — Getter
3. The @x.setter Decorator
4. Computed Properties
5. BankAccount with @property
6. Using BankAccount with Properties
7. @property — The Full Picture



# Outline

## 1. The Problem with Getters and Setters

## 2. The @property Decorator — Getter

## 3. The @x.setter Decorator

## 4. Computed Properties

## 5. BankAccount with @property

## 6. Using BankAccount with Properties

## 7. @property — The Full Picture



# 1. The Problem with Getters and Setters

Traditional getter/setter methods work, but they are verbose:

```
p = Person("Alice", 25)
print(p.get_name())      # getter – verbose
p.set_age(26)            # setter – verbose
```

Python offers a more **Pythonic** solution: the @property decorator.

```
# With @property, this is equivalent:
print(p.name)            # reads like a normal attribute
p.age = 26               # writes like a normal attribute
                          # but validation still runs!
```



# 1. The Problem with Getters and Setters

Traditional getter/setter methods work, but they are verbose:

```
p = Person("Alice", 25)
print(p.get_name())      # getter – verbose
p.set_age(26)            # setter – verbose
```

Python offers a more **Pythonic** solution: the @property decorator.

```
# With @property, this is equivalent:
print(p.name)            # reads like a normal attribute
p.age = 26               # writes like a normal attribute
                        # but validation still runs!
```

@property lets you expose getter/setter logic using **attribute-style access**. The caller doesn't even know validation is happening.



# Outline

1. The Problem with Getters and Setters
2. The @property Decorator — Getter
3. The @x.setter Decorator
4. Computed Properties
5. BankAccount with @property
6. Using BankAccount with Properties
7. @property — The Full Picture



## 2. The @property Decorator — Getter

```
class Person:
    def __init__(self, name, age):
        self._name = name
        self._age = age

    @property
    def name(self):
        """The person's name (read-only in this version)."""
        return self._name

    @property
    def age(self):
        """The person's age."""
        return self._age

p = Person("Alice", 25)
```



## 2. The @property Decorator — Getter

```
print(p.name)      # Alice    (no parentheses – accessed like an attribute)
print(p.age)       # 25

# p.name = "Bob"    # AttributeError – no setter defined yet
```



# Outline

1. The Problem with Getters and Setters
2. The @property Decorator — Getter
- 3. The @x.setter Decorator**
4. Computed Properties
5. BankAccount with @property
6. Using BankAccount with Properties
7. @property — The Full Picture



### 3. The @x.setter Decorator

Define a setter by using `@property_name.setter`.

```
class Person:
    def __init__(self, name, age):
        self._name = name
        self._age = None
        self.age = age      # use the setter even in __init__

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        """Validate and set age."""
        if not isinstance(value, int):
```



### 3. The @x.setter Decorator

```
        raise ValueError("Age must be an integer")
    if value < 0 or value > 150:
        raise ValueError(f"Age must be 0-150, got {value}")
    self._age = value
```

```
p = Person("Alice", 25)
print(p.age)          # 25
p.age = 26             # calls the setter
print(p.age)          # 26

try:
    p.age = -5         # raises ValueError
except ValueError as e:
    print(f"Error: {e}")
```



# Outline

1. The Problem with Getters and Setters
2. The @property Decorator — Getter
3. The @x.setter Decorator
- 4. Computed Properties**
5. BankAccount with @property
6. Using BankAccount with Properties
7. @property — The Full Picture



## 4. Computed Properties

A property can **compute** its value on the fly from other attributes.

```
class Rectangle:
    def __init__(self, width, height):
        self._width = width
        self._height = height

    @property
    def width(self): return self._width

    @property
    def height(self): return self._height

    @property
    def area(self):
        """Computed – no storage needed."""
```



## 4. Computed Properties

```
    return self._width * self._height
```

```
@property
```

```
def perimeter(self):
```

```
    return 2 * (self._width + self._height)
```

```
@property
```

```
def is_square(self):
```

```
    return self._width == self._height
```

```
r = Rectangle(4, 6)
```

```
print(r.area)           # 24 – looks like an attribute, computed from w*h
```

```
print(r.perimeter)     # 20
```

```
print(r.is_square)     # False
```



# Outline

1. The Problem with Getters and Setters
2. The @property Decorator — Getter
3. The @x.setter Decorator
4. Computed Properties
- 5. BankAccount with @property**
6. Using BankAccount with Properties
7. @property — The Full Picture



## 5. BankAccount with @property

```
class BankAccount:
    def __init__(self, account_no, name, initial_balance=0):
        self._account_no = account_no
        self._name = name
        self._balance = 0
        self.balance = initial_balance # use setter

    @property
    def account_no(self):
        return self._account_no # read-only (no setter)

    @property
    def name(self):
        return self._name

    @name.setter
```



## 5. BankAccount with @property

```
def name(self, value):
    if not value.strip():
        raise ValueError("Name cannot be blank")
    self._name = value.strip()

@property
def balance(self):
    return self._balance

@balance.setter
def balance(self, value):
    if value < 0:
        raise ValueError("Balance cannot be negative")
    self._balance = value
```



# Outline

1. The Problem with Getters and Setters
2. The @property Decorator — Getter
3. The @x.setter Decorator
4. Computed Properties
5. BankAccount with @property
- 6. Using BankAccount with Properties**
7. @property — The Full Picture



## 6. Using BankAccount with Properties

```
acc = BankAccount("ACC001", "Alice Murphy", 1000)
```

```
# Attribute-style access – clean and readable
```

```
print(acc.balance)           # 1000
```

```
print(acc.name)              # Alice Murphy
```

```
print(acc.account_no)        # ACC001
```

```
# Set via property (validation runs automatically)
```

```
acc.name = "Alice Smith"
```

```
print(acc.name)              # Alice Smith
```

```
# These raise ValueError:
```

```
try:
```

```
    acc.balance = -500        # caught by setter
```

```
except ValueError as e:
```

```
    print(f"Error: {e}")
```



## 6. Using BankAccount with Properties

```
try:
    acc.account_no = "NEW" # no setter – AttributeError
except AttributeError as e:
    print(f"Error: {e}")
```



# Outline

1. The Problem with Getters and Setters
2. The @property Decorator — Getter
3. The @x.setter Decorator
4. Computed Properties
5. BankAccount with @property
6. Using BankAccount with Properties
- 7. @property — The Full Picture**



## 7. @property – The Full Picture

Three decorators work together:

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):
        """Getter – read the temperature."""
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """Setter – validate before storing."""
        if value < -273.15:
            raise ValueError("Below absolute zero")
```



## 7. @property – The Full Picture

```
self._celsius = value
```

```
@celsius.deleter
```

```
def celsius(self):
```

```
    """Deleter – called when: del obj.celsius"""
```

```
    print("Deleting temperature...")
```

```
    del self._celsius
```

```
@property
```

```
def fahrenheit(self):
```

```
    """Computed read-only property."""
```

```
    return self._celsius * 9/5 + 32
```

```
t = Temperature(100)
```

```
print(t.celsius)           # 100
```

```
print(t.fahrenheit)       # 212.0
```



## 7. @property – The Full Picture

```
t.celsius = 0
print(t.fahrenheit)    # 32.0
del t.celsius           # calls deleter
```



# Thanks for Watching - Any questions?

